CRR Binomial Option Tree

Visual Simulation -- Construction, Animation & Execution Guide

What Is This Animation?

The CRR Binomial Option Tree animation is a step-by-step visual simulation of how the Cox-Ross-Rubinstein (1979) binomial lattice prices an American-style put option on AMD stock. It was generated as part of the CS495 Deep Scholar project to make the abstract mathematics of option pricing tangible for presentation and teaching purposes.

The animation walks through the exact same algorithm used in tree.py inside the project pipeline, running on live AMD option chain data (S=$275.10, K=$280, 11 days to expiry). Rather than showing only the final answer, it reveals how the model builds knowledge of the option value by constructing the price lattice forward through time, then reasoning backwards from expiry to today.

Purpose of the Animation

1. Make backward induction visible

The hardest concept in binomial pricing is backward induction: you cannot know today's option value until you know all future values first. The animation makes this concrete by drawing option values from right to left, one column at a time, so an audience watches the computation propagate toward the present -- exactly as the algorithm executes.

2. Show American early-exercise logic

At every interior node the model compares two quantities: the continuation value (hold the option and wait) vs. the intrinsic value (exercise immediately). Amber-colored nodes show where immediate exercise wins, making it visually clear which stock price levels and time steps trigger early exercise of the AMD put.

3. Validate the project pipeline

The animation uses identical parameters to the production pipeline (config.yaml). The root node option value visible at the end of the animation ($14.87) is the same V_model reported by edge.py and compared against the Fidelity market limit price. Showing the full construction builds confidence in the model output.

4. Presentation and reporting asset

The animated GIF (crr_tree_simulation.gif) loops automatically and requires no additional software -- it opens in any browser, PowerPoint, Keynote, or Google Slides. The companion static PNG (crr_tree_final.png) shows the completed tree and is suitable for insertion into the written research report.

Animation Phases (198 frames, 15 fps, ~13 seconds, loops)

Phase	Title	Frames	Color	Description
Phase 1	Forward Pass	Frames 1-110	Blue	The stock price lattice builds left (today) to right (expiry), one column at a time. Each node shows S(i,j) = S * u^(i-j) * d^j. Up/down edge factors (u=1.0932, d=0.9147) are labeled on the first column. Risk-neutral probabilities p and q appear once the first branches are drawn.
Phase 2	Terminal Payoffs	Frames 111-132	Green / Gray	The rightmost column (step N=4, expiry) transitions from blue to color: green for in-the-money nodes where the put has value (K-S > 0), gray for out-of-the-money nodes. Each node shows both the stock price and V = max(K-S, 0).
Phase 3	Backward Induction	Frames 133-198	Amber / Teal	Option values propagate right to left. At each node: hold = disc*(p*V_up + q*V_dn) vs. exercise = max(K-S, 0). Amber = early exercise optimal. Teal = hold is better. The root node (step 0) reveals the final model price V = $14.87.

Node Color Legend

Blue	Forward pass -- stock price node, option value not yet computed
Green	Terminal node -- in the money (intrinsic value > 0)
Gray	Terminal node -- out of the money (intrinsic value = 0)
Amber	Backward induction -- early exercise is optimal at this node
Teal	Backward induction -- continuation value used (hold is better)

CRR Model Formulas Displayed in the Animation

Up factor u	exp(sigma * sqrt(dt)) = exp(0.65 * sqrt(11/(365*4))) = 1.0932
Down factor d	1 / u = 0.9147
Risk-neutral p	(exp(r*dt) - d) / (u - d) = 0.5086
Discount factor	exp(-r * dt) = exp(-0.053 * 11 / (365*4))
Stock node	S(i,j) = S * u^(i-j) * d^j
American option V	max(max(K-S, 0), disc * (p*V(i+1,j) + q*V(i+1,j+1)))
Root node result	V(0,0) = $14.87 (AMD $280 Put, 11 days, matches pipeline)

Output Files

crr_tree_simulation.gif	496 KB | Animated GIF, 198 frames, 15 fps, loops forever
crr_tree_final.png	234 KB | High-resolution static final frame (150 dpi)
Location	/Users/dewaynedantzler/ (home directory)

How to Run the Animation

Prerequisites

The animation script requires Python 3 with numpy, matplotlib, and Pillow. All three are present in both the project virtual environment (.venv) and the system Anaconda installation already on this machine.

Install Pillow if running outside the project venv:

```
pip install Pillow
```

Option A -- Standalone script (recommended)

Save the animation code as generate_crr_animation.py (see the project repository) and run it from any directory. The GIF and PNG are written to the current folder.

```
python3 generate_crr_animation.py
```

Produces two files in the working directory:

```
crr_tree_simulation.gif    # animated simulation, loops automatically
crr_tree_final.png         # static final frame for slides / report
```

Option B -- From the project repo using the virtual environment

```
cd ~/Documents/Bellevue\ College/Spring\ 2026/CS495/GitHubRepo/CS495-Deep-Scholar
source .venv/bin/activate
python3 generate_crr_animation.py
```

Option C -- Inside a Jupyter notebook

```
from IPython.display import Image
%run generate_crr_animation.py
Image('crr_tree_simulation.gif')   # displays inline in the notebook
```

Customising the Animation Parameters

Edit these variables at the top of generate_crr_animation.py:

S0	Current stock price	(default: 275.10)
K	Strike price	(default: 280.0)
r	Risk-free rate	(default: 0.053)
sigma	Implied volatility	(default: 0.65)
T_days	Calendar days to expiry	(default: 11)
N	Binomial tree steps	(default: 4 -- increase for more detail)
HOLD	Frames per column	(default: 12 -- reduce to speed up)

Viewing the Output

macOS Finder	Double-click crr_tree_simulation.gif -- opens and plays in Safari
Any browser	Drag the .gif into a browser tab -- plays and loops automatically
PowerPoint	Insert > Pictures > select .gif -- animates in slideshow mode
Google Slides	Insert > Image > Upload -- animates automatically during presentation
Keynote	Drag .gif onto a slide, set 'Original Size' -- plays on click

Relationship to the Project Pipeline

The animation is a companion visualisation to the main pipeline and does not need to be run as part of 'make run'. The production pipeline in project/main.py uses N=100 steps for pricing accuracy; the animation uses N=4 for visual readability. The root node value shown at the end of the animation matches the V_model reported in project/outputs/edges.csv for Ticket 4 (Buy to Open AMD Put $280 = $14.87).

Animation step	Pipeline file	What it maps to

Forward pass	`tree.py`	stock[] lattice -- S * u^(i-j) * d^j (lines 28-30)
Terminal payoffs	`tree.py`	V = max(K-S,0) terminal assignment (lines 32-35)
Backward induction	`tree.py`	V = max(intrinsic, V_hold) loop (lines 37-50)
Amber node	`tree.py`	exercise_boundary[] tracking (lines 52-55)
Root node value	`edge.py`	V_model used in compute_edges()
Root node value	`kelly.py`	feeds into kelly_fractions() via edge